RAPPORT DE TRAVAUX PRATIQUES

ABSTRACT FACTORY

Création de familles d'objets en Java

Étudiant : Togyanouba Moïse

Niveau : Master 1

Matricule : XXXXXXX

Lien Github : https://github.com/organizations/INF-461/settings/profile

Unité d'enseignement : SIGL

Code : INF 461

Établissement : Université de Yaoundé 1

Année académique : 2026- 2027

Projet réalisé en langage Java

1. Introduction

Le patron Abstract Factory permet de créer des familles d'objets liés sans que le programme client connaisse les classes concrètes utilisées. Il fournit une interface commune de création pour les différents produits d'une même famille.
Dans ce projet, les produits sont les scooters et les automobiles. Chaque famille possède une catégorie de motorisation : essence ou électrique dans la version du cours.

2. Structure du patron

Élément	Responsabilité
IProduitFactory	Interface qui déclare les méthodes de création d'un scooter et d'une automobile.
ProduitFactory1	Fabrique concrète qui crée les produits à essence.
ProduitFactory2	Fabrique concrète qui crée les produits électriques.
Scooter	Classe abstraite représentant le type de produit Scooter.
Automobile	Classe abstraite représentant le type de produit Automobile.
Client	Utilise une fabrique et manipule les produits à travers les types abstraits.

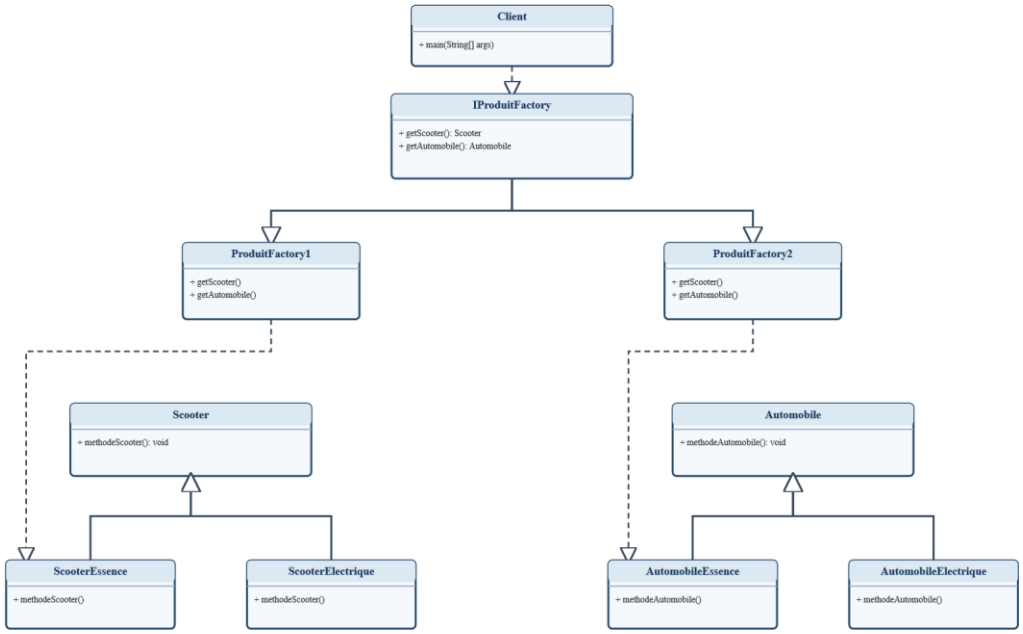
3. Version du cours

La version originale contient deux fabriques concrètes. ProduitFactory1 crée un scooter à essence et une automobile à essence. ProduitFactory2 crée un scooter électrique et une automobile électrique.

Les classes ScooterEssence, ScooterElectrique, AutomobileEssence et AutomobileElectrique héritent respectivement de leurs classes abstraites. Le client choisit une fabrique et utilise les objets retournés sans dépendre directement de leurs classes concrètes.

4. Diagramme UML de la version du cours

ABSTRACT FACTORY - DIAGRAMME UML



Trait pointille : dependance / creation. Triangle blanc : heritage ou realisation.

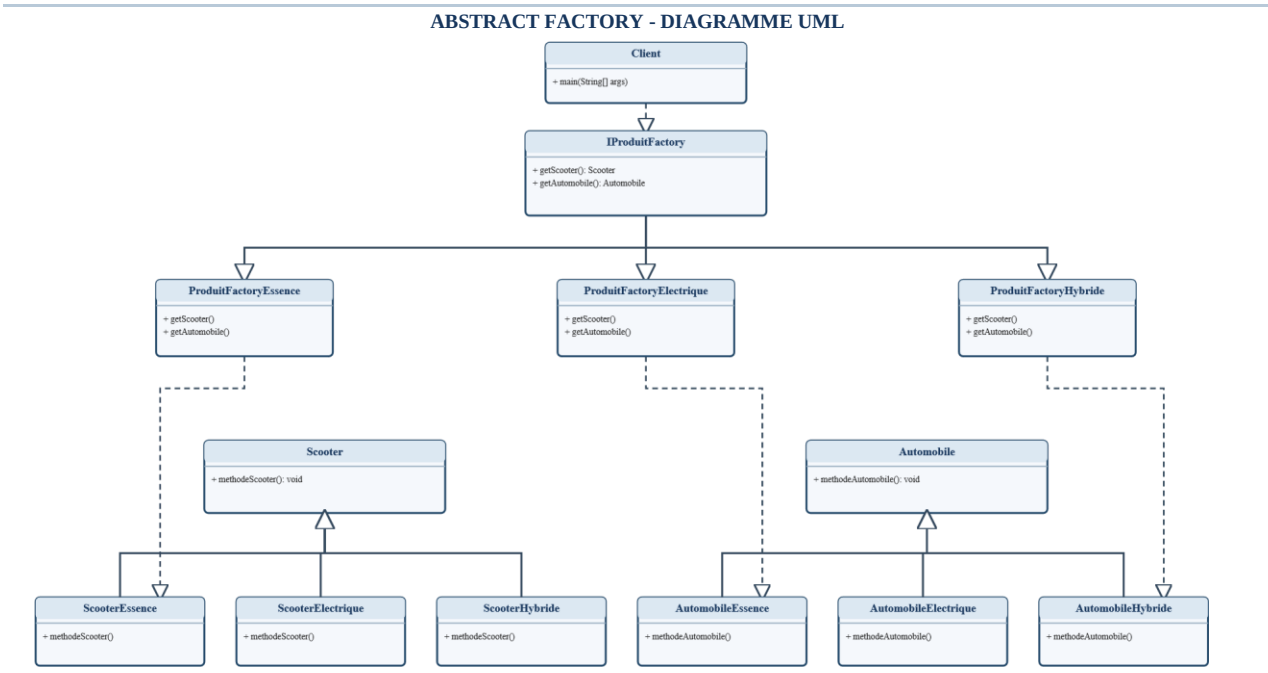
5. Modification apportée

La version modifiée ajoute une troisième famille de motorisation : hybride. Deux nouvelles classes concrètes sont créées : ScooterHybride et AutomobileHybride.

Une nouvelle fabrique, ProduitFactoryHybride, implémente l'interface IProduitFactory. Elle fournit les deux produits hybrides. L'interface n'a pas besoin d'être modifiée, car les méthodes getScooter() et getAutomobile() couvrent déjà les deux types de produits.

Élément	Version du cours	Version modifiée
Catégories	Essence et électrique	Essence, électrique et hybride
Fabriques concrètes	ProduitFactory1 et ProduitFactory2	Trois fabriques spécialisées par motorisation
Produits concrets	Quatre classes	Six classes
Interface abstraite	IProduitFactory	Conservée sans modification

6. Diagramme UML de la version modifiée



Trait pointille : dependance / creation. Triangle blanc : heritage ou realisation.

7. Tests et résultats

Les deux versions ont été compilées avec javac -encoding UTF-8 *.java puis exécutées à partir de la classe Client.

Version	Tests réalisés	Résultat attendu
Cours	Fabrique essence et fabrique électrique	Création et affichage des deux produits de chaque famille.
Modifiée	Essence, électrique et hybride	Création et affichage des six produits concrets.

8. Analyse

Le client dépend de l'interface IProduitFactory et des classes abstraites Scooter et Automobile. Il n'a pas besoin de connaître les classes concrètes instanciées par chaque fabrique.

L'ajout de la catégorie hybride montre l'intérêt du patron : une nouvelle fabrique concrète et de nouveaux produits peuvent être ajoutés sans changer l'interface commune ni la logique générale du client.

9. Conclusion

Ce travail a permis de mettre en œuvre le patron Abstract Factory en Java. La version du cours crée deux familles de produits : essence et électrique. La version modifiée ajoute la famille hybride. Les deux versions ont été compilées et exécutées.